ML and NLP with Python

Python Libraries

- **NumPy** **Numerical computing, arrays**
- **Pandas** **Data manipulation**
- **Matplotlib** **Data visualization**
- **Seaborn** **Statistical data visualization**
- **Scikit-Learn** **Machine learning algorithms**
- TensorFlow Deep learning, neural networks
- Keras High-level API for deep learning (built on top of TensorFlow)
- PyTorch Deep learning (research-focused)
- XGBoost Gradient boosting for structured data
- LightGBM Fast boosting algorithm
- OpenCV Computer vision and image processing
- **NLTK** **Natural language processing**
- SpaCy Advanced NLP

scikit

- scikit-learn (sklearn) is a powerful machine learning library in Python that provides tools for:
  - Data Preprocessing (handling missing data, scaling, encoding)
  - Feature Extraction (Bag of Words, TF-IDF, PCA)
  - Supervised Learning (Regression & Classification models)
  - Unsupervised Learning (Clustering, Anomaly Detection)
  - Model Selection & Evaluation (Cross-validation, Hyperparameter tuning)

- Task 1: Load & Explore a Dataset

```
import pandas as pd
df = pd.read_csv('data.csv')  # Load dataset
print(df.head())  # Show first 5 rows
print(df.info())  # Dataset summary
print(df.describe())  # Statistical summary
```

- Task 2: Train-Test Split

```
from sklearn.model_selection import train_test_split
X = df.drop('Target', axis=1)  # Features
y = df['Target']  # Labels
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- Task 3: Linear Regression

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)  # Train model
y_pred = model.predict(X_test)  # Make predictions
```

The random_state parameter ensures that the data split is reproducible. It controls the randomness of the train-test split, meaning:
Same random_state → Same Split Every Time
Different random_state → Different Split Every Time

- Task 4: Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
clf = LogisticRegression()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

NLP in Python

- Learn how unstructured human language can be converted into structured numerical data so that a computer can process it.
  - Computers work with: Numbers, Vectors, Matrices
  - Humans: Text

So, NLP converts

- Text → structured representation → numerical representation → computation → interpretation

Experiment 1: Basic Text Preprocessing in Python

```
text = """” Natural Language Processing is an important area of Artificial Intelligence.
It enables computers to process and understand human language.
NLP is used in chatbots, search engines and recommendation systems."""
```

What type of data is this?
Answer: Unstructured text.

```
print(text)
print(len(text))
print(text.lower())

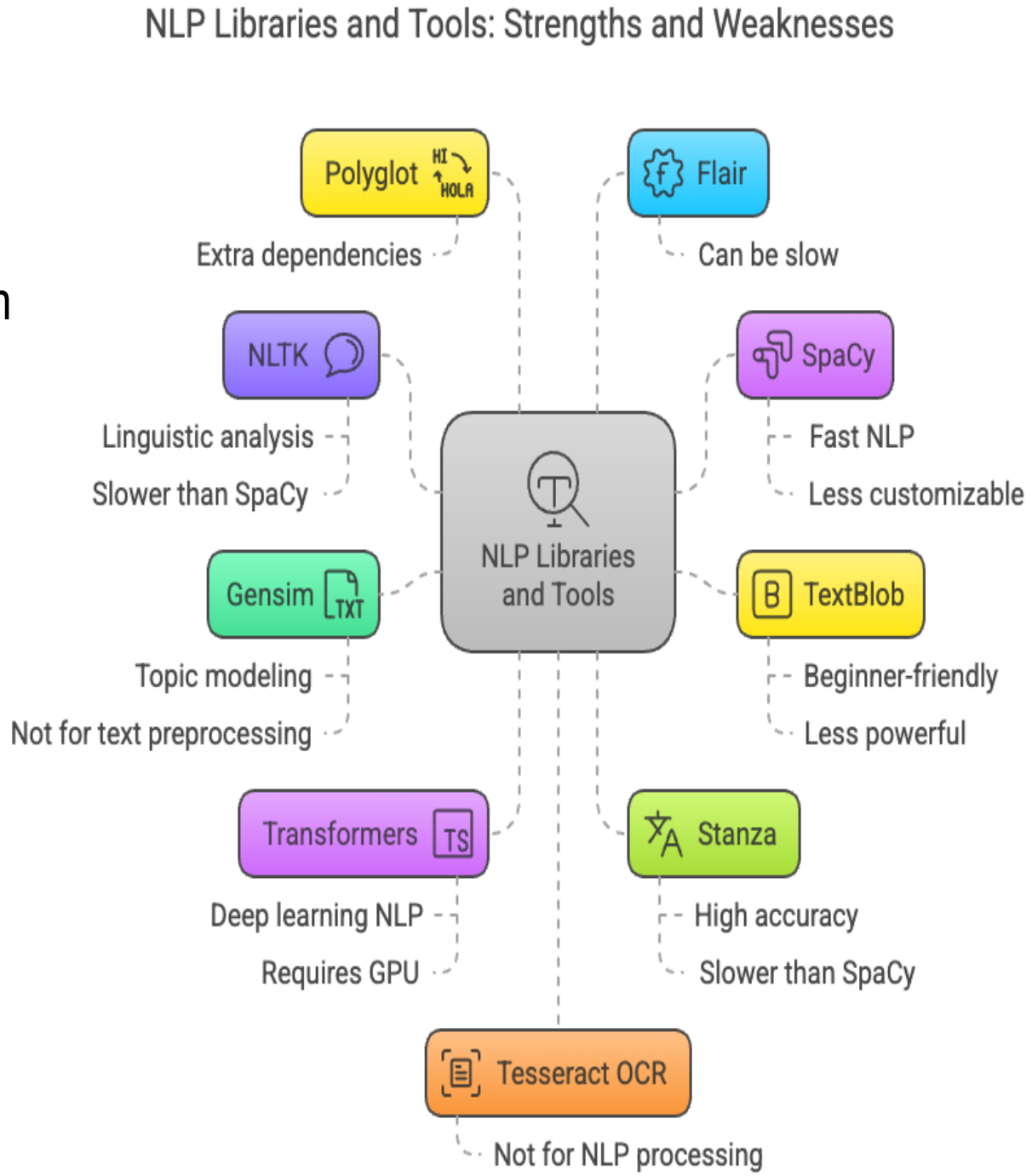
words = text.split()

print(words)
print(len(words))
```

['Natural', 'Language', 'Processing', 'is', 'an', 'important', 'area', 'of', 'Artificial', 'Intelligence.', 'It', 'enables', 'computers', 'to', 'process', 'and', 'understand', 'human', 'language.', 'NLP', 'is', 'used', 'in', 'chatbots,', 'search', 'engines', 'and', 'recommendation', 'systems.’]
29

NLP Libraries in Python

- Python has number of libraries for NLP to perform tokenization, sentiment analysis, machine translation, text summarization, and more.
  - **NLTK (Natural Language Toolkit)**
  - **spaCy**
  - **TextBlob**
  - **Transformers (by Hugging Face)**
  - **Gensim**
  - **Tesseract OCR (for Text Extraction from Images)**
  - **Polyglot**
  - **Keras (for deep learning NLTK)**



Text Preprocessing in Python

- Text Cleaning/Tokenization using Python RegEx Module
- Regular Expressions - Sequence of characters that defines a search pattern. It is commonly used for:
    - Finding specific patterns in text (e.g., emails, dates, phone numbers).
    - Replacing or cleaning text (e.g., removing special characters).
    - Splitting text into meaningful components.
- Python has a built-in module named “re” that is used for regular expressions in Python

RegEx - Example

```
import re
s = “CognitiveComputing: A computer science subject for geeks”
match = re.search(‘subject', s)
print('Start Index:', match.start())
print('End Index:', match.end())
```

Output:
Start Index: 39
End Index: 46

re.findall() - finds and returns all matching occurrences in a list

```
import re
string = """Hello my Number is 987654321 and
                my friend's number is 123456789"""
regex = r'\d+'
match = re.findall(regex, string)
print(match)
```

Output:
['987654321', '123456789']

Here r character (r'portal') stands for raw, not regex. The raw string is slightly different from a regular string, it won't interpret the \ character as an escape character. This is because the regular expression engine uses \ character for its own escaping purpose.

Other Regex Functions

re.compile()	**Regular expressions are compiled into pattern objects**
re.split()	**Split string by the occurrences of a character or a pattern.**
re.sub()	**Replaces all occurrences of a character or patter with a replacement string.**
re.escape()	**Escapes special character**
re.search()	**Searches for first occurrence of character or pattern**

Split() for word tokenization

text = "There are multiple ways we can perform tokenization on given text data. We can choose any method based on langauge, library and purpose of modeling."

Split text by whitespace

tokens = text.split()

print(tokens)

['There', 'are', 'multiple', 'ways', 'we', 'can', 'perform', 'tokenization', 'on', 'given', 'text', 'data.', 'We', 'can', 'choose', 'any', 'method', 'based', 'on', 'langauge,', 'library', 'and', 'purpose', 'of', 'modeling.']

Tokenization using NLTK Tokenizer

- NLTK provides several built-in tokenizers for different NLP tasks.
  1. Sentence Tokenization (sent_tokenize)
  2. Word Tokenization (word_tokenize)
  3. Regular Expression Tokenizer (RegexpTokenizer) - Custom regex-based tokenization
  4. White Space Tokenizer (WhitespaceTokenizer)
  5. WordPunct Tokenizer (WordPunctTokenizer)
  6. Tweet Tokenizer (TweetTokenizer)
  7. SyllableTokenizer

Experiment 2: Tokenization with NLTK

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab’)
from nltk.tokenize import sent_tokenize, word_tokenize
text = "NLP is interesting. It helps computers process language."
print(sent_tokenize(text))
print(word_tokenize(text))
```

punkt and punkt_tab provide tokenizer resources (algorithms for split) required by NLTK for sentence and word tokenization.

```
['NLP is interesting.', 'It helps computers process language.’]
['NLP', 'is', 'interesting', '.', 'It', 'helps', 'computers', 'process', 'language', '.']
```

Steps before using NLTK in Jupyter Notebook

%pip import nltk

install nltk

print(nltk.__version__)

```
import os
print(os.getcwd())
import os
nltk_path = os.path.expanduser("drive/nltk_data/tokenizers")
os.makedirs(nltk_path, exist_ok=True)
print(f"Created directory: {nltk_path}")
import nltk
nltk.data.path.append(os.path.expanduser("drive/nltk_data"))
print("NLTK path updated!")
import zipfile
nltk_path = os.path.expanduser("drive/nltk_data/tokenizers")
with zipfile.ZipFile("punkt.zip", "r") as zip_ref:          // NLTK requires the punkt tokenizer model
    zip_ref.extractall(nltk_path)
print("Extracted `punkt` successfully!")
```

Word Tokenization

- from nltk.tokenize import word_tokenize
- text = """There are multiple ways we can perform tokenization on given text data. We can choose any method based on langauge, library and purpose of modeling."""
- tokens = word_tokenize(text)
- print(tokens)

['There', 'are', 'multiple', 'ways', 'we', 'can', 'perform', 'tokenization', 'on', 'given', 'text', 'data', '.', 'We', 'can', 'choose', 'any', 'method', 'based', 'on', 'langauge', ',', 'library', 'and', 'purpose', 'of', 'modeling', '.']

Sentence Tokenization

- from nltk.tokenize import sent_tokenize
- text = """Characters like periods, exclamation point and newline char are used to separate the sentences. But one drawback with split() method, that we can only use one separator at a time! So sentence tonenization wont be foolproof with split() method."""
- sent_tokenize(text)

['Characters like periods, exclamation point and newline char are used to separate the sentences.’,
'But one drawback with split() method, that we can only use one separator at a time!’,
'So sentence tonenization wont be foolproof with split() method.']

Split() for sentence tokenization

text = """Characters like periods, exclamation point and newline char are used to separate the sentences. But one drawback with split() method, that we can only use one separator at a time! So sentence tonenization wont be foolproof with split() method."""

text.split(". ")

Note the space after the full stop makes sure that we dont get empty element at the end of list.

Advantage: ['Characters like periods, exclamation point and newline char are used to separate the sentences'

Drawbacks of using split() for sentence tokenization

- We can only use one separator at a time!
- Cannot handle different sentence-ending punctuation
  - Hello! How are you? I am fine.
  - text.split(".")
  - it won't split at ! or ?.
- Problem with abbreviations
  - Dr. Sharma teaches NLP. He is from India.
- Decimal numbers cause problems
  - The accuracy is 95.5%. It is good.

split() treats punctuation as a delimiter, whereas a sentence tokenizer uses linguistic rules to determine where a sentence actually ends.

Exp 3: Text Normalization

Lowercasing, StopWord Removal, Stemming, Lemmatization, PoS Tagging, Token vs Type, Zipf’s law

Lowercasing & Punctuation Removal

```
text = "Natural Language Processing is AMAZING!”
text = text.lower()
print(text)
```

Output: natural language processing is amazing!

```
import string
text_without_punctuation = text.translate( str.maketrans('', '', string.punctuation) )
print(text_without_punctuation)
```

Output: natural language processing is amazing

Do we always remove punctuation?

- It depends on the **NLP task**.
- For something like **document similarity or basic word-frequency analysis**, removing punctuation may be useful.
- But for **sentiment analysis, emotion detection, dialogue, question detection, or some language-understanding tasks**, punctuation may be useful
- Preprocessing is task-dependent. We do not remove punctuation simply because it is punctuation; we remove it when it is not useful for the particular NLP task.

StopWord Removal

- from nltk.corpus import stopwords
- *# Get English stopwords*
- stop_words = set(stopwords.words("english"))
- print(stop_words) # Display some stopwords

- stop_words.add("example") # Adding "example" to stopwords list
- stop_words.remove("not") # Removing "not" (if negation is important)

Removing StopWord from Sentence

```
from nltk.tokenize import word_tokenize

text = "This is a simple example to demonstrate the removal of stopwords in NLP."

# Tokenizing the text

tokens = word_tokenize(text)

# Remove stopwords

filtered_tokens = [word for word in tokens if word.lower() not in stop_words]

print(filtered_tokens)
```

Stemming

- RegexpStemmer - **custom stemming rules** using **regular expressions (regex) (regex vs RegexpStemmer)**
  - **Regex -** built-in re module for **general-purpose pattern-matching**
    - Works on arbitrary text
  - **RegexpStemmer** - an **NLP stemming tool** built using regular-expression patterns.
    - regex rules specifically to reduce words

```
import re

text = "I have 25 apples and 10 oranges."

numbers = re.findall(r'\d+', text)
print(numbers)
```

```
from nltk.stem import RegexpStemmer

stemmer = RegexpStemmer(r'ing$')

words = ["playing", "running", "walking"]

for word in words:
    print(stemmer.stem(word))
```

RegexpStemmer

```
from nltk.stem import RegexpStemmer
# Define a regex pattern to remove common suffixes like "ing", "ed", "es"
regexp_stemmer = RegexpStemmer(r"ing$|ed$|es$")
words = ["running", "flies", "studies", "happiness", "played", "jumps"]
stemmed_words = [regexp_stemmer.stem(word) for word in words]
print(stemmed_words)
```

Output:
['runn', 'fli', 'studi', 'happiness', 'play', 'jumps']

Stemming

Other stemmers:

- PorterStemmer
- LancasterStemmer
- SnowballStemmer – Supports multiple languages

PorterStemmer

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
ps = PorterStemmer()
sentence = "Programmers program with programming languages"
words = word_tokenize(sentence)
for w in words:
    print(w, " : ", ps.stem(w))
```

Programmers : programm
program : program
with : with
programming : program
languages : languag

Lemmatization

- The WordNetLemmatizer in NLTK uses the WordNet lexical database to find the base form of words.

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
words = ["running", "flies", "studies", "better", "happily", "geese"]
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
print(lemmatized_words)
```

Output: ['running', 'fly', 'study', 'better', 'happily', 'goose']

TRY YOURSELF: Lemmatization with POS (Part of Speech) Tags

Lemmatization

```
# import these modules
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print("rocks :", lemmatizer.lemmatize("rocks"))
print("corpora :", lemmatizer.lemmatize("corpora"))
# a denotes adjective in "pos"
print("better :", lemmatizer.lemmatize("better", pos="a"))
```

```
rocks : rock
corpora : corpus
better : good
```

POS Tagging

```
nltk.download('averaged_perceptron_tagger_eng’)
from nltk import pos_tag
sentence = "The intelligent student solved the difficult problem."
tokens = word_tokenize(sentence)
tags = pos_tag(tokens)
for word, tag in tags:
    print(word, tag)
```

The DT
intelligent JJ
student NN
solved VBD
the DT
difficult JJ
problem NN

Do it Yourself!

- Token vs Type
- Word Frequency
- Zipf’s Law
- Singularize and Pluralize text using TextBlob
- TextBlob: Translate a sentence from Spanish to English

N-gram Tokens

- from nltk.util import ngrams token
- s = ["machine", "learning", "is", "very", "useful"]
- unigrams = list(ngrams(tokens, 1))
- bigrams = list(ngrams(tokens, 2))
- trigrams = list(ngrams(tokens, 3))
- print("Unigrams:", unigrams)
- print("Bigrams:", bigrams)
- print("Trigrams:", trigrams)